

Module- 2

DYNAMIC AND FUNCTIONAL MODELING

1

Module 2

Dynamic modeling: Events and States, Operations, Nested state diagrams, Concurrency, Advanced Dynamic Modeling Concepts, A sample Dynamic Model, Relationship of Object and Dynamic models.

Functional modeling: Functional models, Data Flow Diagrams, Specifying Operations, Constraints, A sample Functional Model

2

DYNAMIC MODELING

- The **dynamic model** shows the **time-dependent behaviour** of the **system and the objects in it**.
- A system can be understood by first examining its **static structure**, that is, the **structure of its objects and relationship to each other at a single moment in time**.
- Then we examine changes to the objects and their relationship over time. Those **aspects of a system that are concerned with time and changes are the dynamic model**.
- The **dynamic model** is a **collection** of **state diagrams** that **interact** with each other via **shared events**.

3

EVENTS

- An **event** is **something that happens at a point in time**.
- We can understand an event as some action oriented result, such as **mouse click**. Whenever you will click on a mouse, the appropriate **action** takes place.
- An event has **no duration**.
- An event is simply an occurrence.
- An event is actually a **one way transmission of information from one object to another**, but some times it may be that an event that occurs on a single object and changes the state of that object.

4

- Two **events** can take place at the **same time**, **one after the other**, or both the events **independently** of each other and occurring **simultaneously**. (causally related-before ,after)

•Eg: Two trains can depart at the same time for two different places, or two can depart from the same place, but one after the other. It means that the two events can be independent as well as dependent on each other.

- Two events which are **unrelated** and occur at the **same time** are known as **concurrent** events. They have no effect on each other.

•We cannot find particular order between the two events, because they can occur in any order.

5

- An object sending an event to another object may expect a reply, but the reply will be a **separate event** of the second object. So, you may see conversations between two objects as a combination of two or more events.

• Every event is a unique occurrence, but we **group** them into **event classes** and give each event class a name to indicate common structure and behavior

•Eg: Events like train departs which has the attributes train number, class, city, etc. It is not necessary that all the attributes of objects contribute to attributes of events.

Mouse button clicked (left click, location)

Digit dialled (digit)

Phone receiver lifted.

- Events include **error conditions** as well as **normal occurrences**. Eg: motor jammed, transaction aborted, and time out are typical error events.

6

SCENARIOS AND EVENT TRACES

- A **scenario** is a **sequence of events** that occurs **during one particular execution of a system**.

•It may include **all events** in the system, or **only some events** from some selected objects involved in the event.

7

SCENARIO FOR A PHONE CALL

Caller lifts receiver

Dial tone-begins.

Caller dial digits

Dial tone ends

Caller dial digit(5)

Caller dial digit(5)

Caller dial digit(1)

Caller dial digit(2)

Caller dial digit(3)

Caller dial digit(4)

Called phone beings ringing

Ring tone appears in calling phone

Called party answers

Called phone stops ringing

Ring tone disappears in calling phone

Phones are connected

Called party hangs up

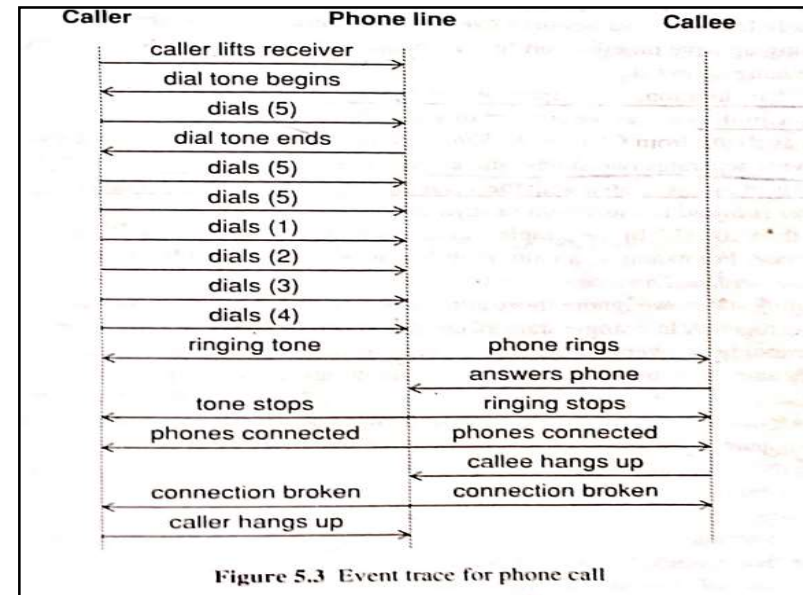
Phones are disconnected

Caller hangs up

8

- Each event **transmits information** from one object to another object. For example, dial tone begins transmits a signal from the phone line to caller.
- The next step after writing the scenario is to **identify the sender and receiver objects of each event**.
- The sequence of events and the objects exchanging events can both be shown in an augmented scenario called an **event trace diagram**.
- This diagram shows each **objects** as **vertical line** and each **event** as **horizontal arrow from sender object to receiver object**.

9



STATES

- A state specifies the response of the object to input values.
- A state corresponds to the **interval between two events received by an object**. Event represents points in time; **states represent intervals of time**.
- For example, after the receiver is lifted and before the first digit is dialed, the phone line is in state dial tone.
- A state **has duration**. It **occupies an interval of time**.
- A state is often **associated with a continuous activity** such as the ringing of telephone or an activity that takes time to complete, such as flying from Chicago to San Francisco.

11

STATE DIAGRAMS

- A state diagram **relates events** and **states**.
- When an event is received, the **next state** depends on the **current state as well as the event**.
- A **change of state caused by an event** is called a **transition**.
- A state diagram is a **graph** whose **nodes** are **states** and whose **directed arcs** are **transitions labeled by event names**.

12

NOTATION

• **Initial State:** This shows the **starting point or first activity** of the flow. It is denoted by a **solid circle**. This is also called a **pseudo state**, where the state has **no variables** describing its further and no activities, to be done.

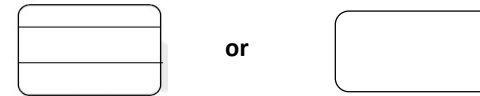


• A state is shown as a **rectangle with rounded corners**. It may have **one or more compartments**. The compartments are **all optional**.

• Name in compartment, holds the (optional) name of the state as a string.

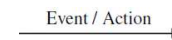
13

State: Represents the state of an object at an instant of time. In a state diagram, there will be multiples of such symbols, one for each state of the object, denoted by a rectangle with rounded corners and compartments.



or

Transition: An arrow indicating the object transition from one state to the other.

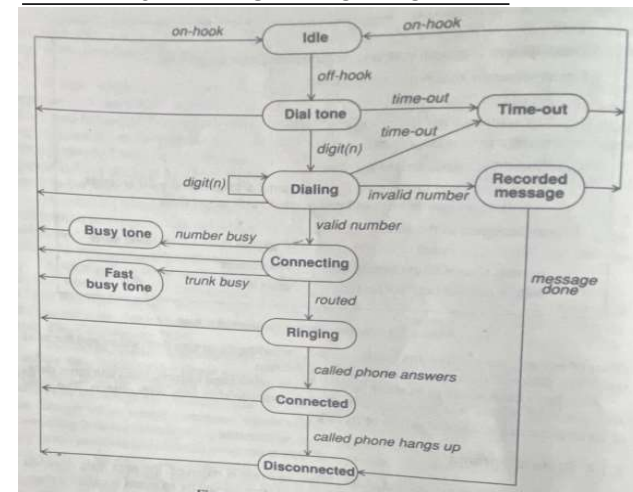


14

Final State: The **end** of the state diagram is shown by a **bull's eye symbol**, also called a **final state**. A final state is another example of a pseudo state because it does not have any variable or action described.



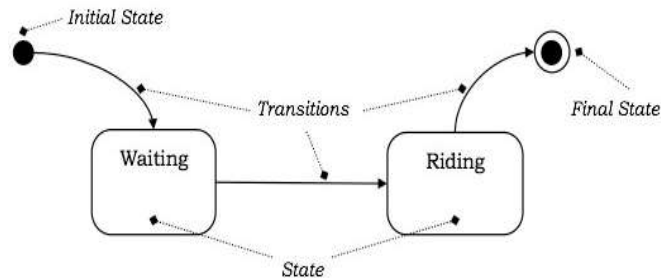
15

EXAMPLE: STATE DIAGRAM FOR PHONE LINE

16

EXAMPLE FOR STATE TRANSITION

- Suppose a person is taking a taxi from place X to place Y.
- The states of the person may be: Waiting (waiting for taxi), Riding (he has got a taxi and is travelling in it), and Reached (he has reached the destination).



17

Conditions

- A condition is a **Boolean function of object values**
- Eg: "the temperature is below freezing "
- A condition is **valid over an interval of time**
- Eg: "the temperature was below freezing from November 15, 1921 until March 3, 1922"
- A **state** can be **defined in terms of a condition**; conversely, **being in a state is a condition**.

18

- Conditions can be used as **guards on transitions**.
- A **guarded transition fires when its event occurs**, but only if the guard condition is **true**.
- Eg: "when you go out in the morning (event), if the temperature is below freezing (condition), then put on your gloves (next state)."
- A guard condition on a transition is shown as a **Boolean expression in brackets following the event name**.
- Eg: A traffic light at an intersection may change only if cars are waiting on the road.

19

OPERATIONS

Controlling Operations

- **Operations** attached to states or transitions are **performed in response** to the corresponding **states or events**.
- **Activity** is an **operation that takes time to complete**.
- An activity is associated with a **state**.
- **Activities** include **continuous operation**, such as displaying a picture on television screen or performing a computation.
- A **state may control continuous activity** such as telephone ringing bell, that persists until an event terminates it by causing a transition from the state.

20

OPERATIONS

Controlling Operations

- The notation '**do:A**' within a state box indicates that **activity A starts on entry to the state and stops on exit.**
- An **action** is an **instantaneous operation.**
- An action is **associated** with an **event**.
- An action represents an **operation** whose **duration** is **insignificant** compared to the resolution of the state diagram.

21

- Actions can also **represent internal control operations**, such as setting attributes or generating other events. (eg: internal counter increments on event)
- The notation for an **action** on a transition is a **slash '/'** and the **name** (or description) of the **action** following the **name** of the **event** that causes it.

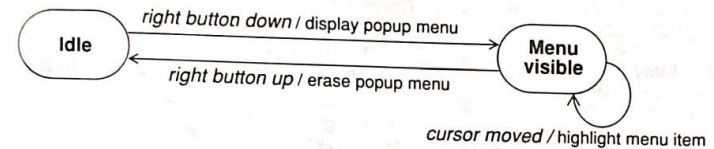


Figure 5.8 Actions for pop-up menu

- When the right button is depressed, the menu is displayed
- When the right button is released, the menu is erased.
- While the menu is visible, the highlighted menu item is updated whenever the cursor moves.

22

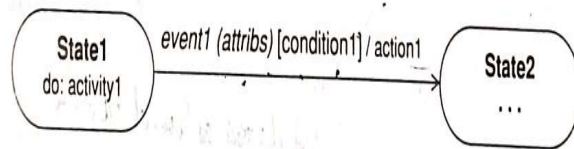


Figure 5.9 Summary of notation for unstructured state diagrams

- A state name is written in boldface within a rounded box.
- An event name is written on a transition arrow and may optionally be followed by one or more attributes within parentheses.
- A condition may be listed within square brackets after an event name.
- An activity is indicated within a state box by the keyword "do:" followed by the name or description of the activity.
- An action is indicated on a transition following the event name by a '/' character followed by the event name.
- All these constructs are optional in state diagrams.

23

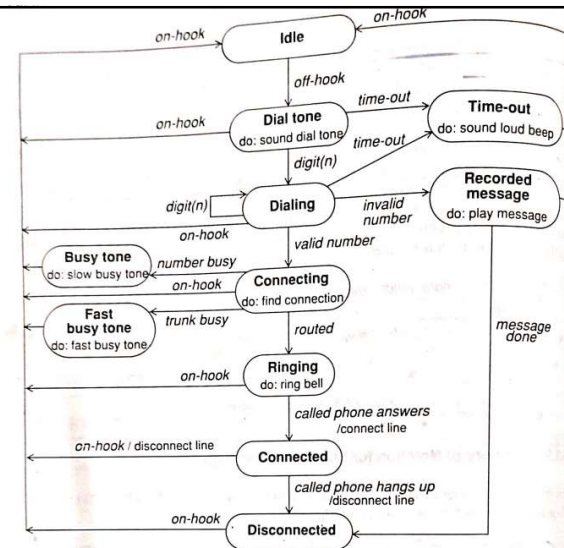


Figure 5.10 State diagram for phone line

24

Need of Nested State Diagrams

State diagrams

- lack expressive power
- are impractical for large problems

25

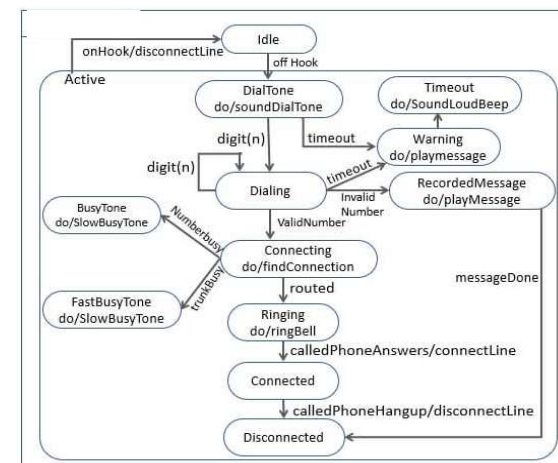
NESTED STATE DIAGRAM

- A **nested state diagram** is used to model the complex system as the regular state diagram is inadequate in describing the large and complex problem.
- A complex system has much **redundancy** and this redundancy can be **resolved** by restructuring the complex system.
- One method to restructure complex system is to **expand** the **state** that has certain sub states.

26

- ▶ Activities in states are composite items denoting other lower-level state diagrams
- ▶ A lower-level state diagram corresponds to a sequence of lower-level states and events that are invisible in the higher-level diagram.

27



Nested state diagram for a phone line

28

- **Active** state is the composite state.
- The states DialTone, Dialing, BusyTone and so on all are the nested state enclosed in a composite Active state.
- You are allowed to nest the states to an arbitrary length.
- Every nested state inside the composite Active state when receives the event '*onHook*', the corresponding state transits to the **Idle** state.

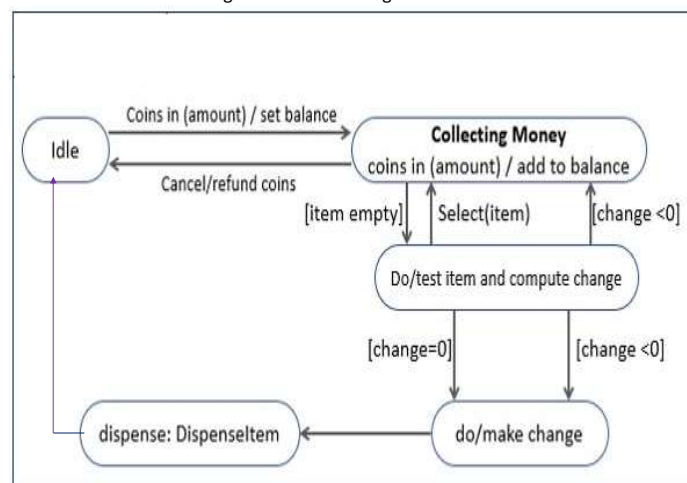
29

Example: Vending Machine

- At the beginning the machine is idle .A person comes and insert some coins into the vending machine and select an item. If the selected item is not available or the amount inserted is insufficient then the machine would wait for the person to take another action.
- If the amount is sufficient and the item is available then the machine would dispense the selected item and deduct the required amount from the inserted amount and would return the remaining change. The diagram below shows the state diagram of the vending machine.

30

Vending Machine State diagram



31

- The state *dispense* may have multiple substates so it is expanded in the lower level state diagram, which is also termed as **submachine**.
- Submachine is defined as a **state diagram which is triggered by another high-level state diagram**.



Figure 5.13 Dispense item activity of vending machine

32

•Events can also be expanded into subordinate state diagrams.

•The select item event actually involves several low-level events.

•The customer keys in an item number and can start over by hitting clear; the selection is confirmed by hitting enter. The label on the bull's-eye indicates the event generated

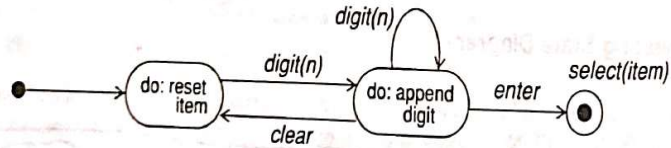


Figure 5.14 Select item transition of vending machine

33

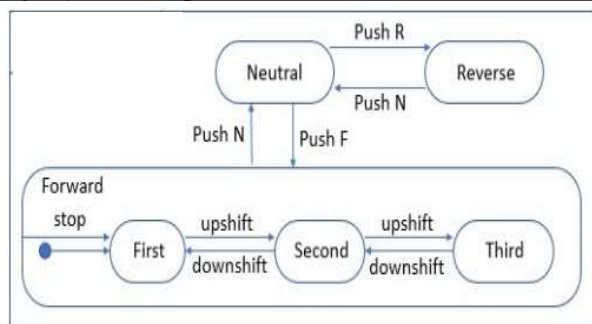
When a state has multiple sub states then they could be nested and enclosed in a contour and could be labelled as the composite state name.

STATE GENERALIZATION

- A nested state diagram is a form of generalization on states.
- State generalization refers to the relationship between a super state and its sub state.
- It's a form of inheritance, where the behavior of the super state is inherited by its sub states.

34

Example(state diagram for automatic transmission)



- The transmission can be in reverse, neutral, or forward states.
- If it is in forward, it can be in first, second or third gear.
- States first, second and third are the sub states of state Forward

35

- The generalization notation for states is different from that used for classes, to avoid a large number of lines that could be confused with transactions.
- A super state is drawn as a large rounded box enclosing all of its sub states.

36

EVENT GENERALIZATION

- Events can be organized into a **generalization hierarchy with inheritance of event attributes**.

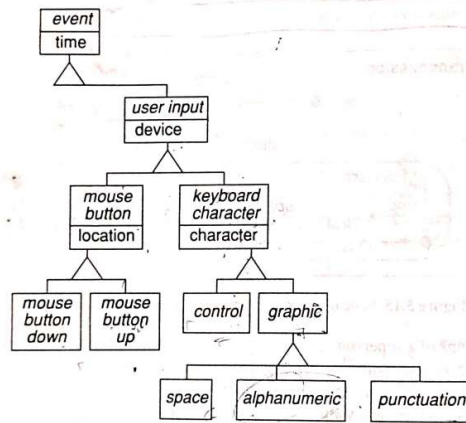


Figure 5.16 Partial event hierarchy for keyboard events

- Above figure shows parts of tree of input events for a workstation.
- Events *mouse button* and *keyboard character* are two kinds of user input.
- Both events inherit attribute *time* from event *event* (the root of the hierarchy) and attribute *device* from event *user input*.
- Mouse button down* and *mouse button up* inherit *location* from *mouse button*.
- Keyboard characters be divided into control characters and graphic characters.
- Ultimately every actual event be as a leaf on a generalization tree of events.
- Inherited event attributes are shown in the second pan of each box.
- An input event triggers transitions on any ancestor event type
Eg.: typing an 'a' would trigger a transition on event alphanumeric as well as event